# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

| Question Type | Focus Area | Questions | Marks |
|---|---|---|---|
| SAST SIMULATION | Creative Things and concept grasping | Q1 | Q1 –100 |
| | GRAND TOTAL: | 100 Marks | |

## Department of Computer Science and Engineering

## ASSESSMENT TOOL 1 – SAST SIMULATION

| | | | |
|---|---|---|---|
| **Course Code:** | CSA5802 | **Course Title:** | DEVSECOPS ENGINEERING AND APPLICATION SECURITY |
| **CO Assessed:** | CO3 – Precision (BL1, BL2, BL3) | **Assessment:** | Assessment Tool 1 |
| **Weightage:** | 30% of CIA | **Total Marks:** | 100 |
| **CO3 Statement:** | Simulate Static Application Security Testing (SAST) using synthetic data | | |
| **Student Name:** | **SHAIK THOUHID** | **Reg. No.:** | **192421280** |
| **Section / Batch:** | **2024** | **Date:** | **10/8/26** |

**Instructions to Students**

- Ensure all diagrams and workflow illustrations are **original, clear, and professionally formatted**.
- Include **all editable source files** (such as .yml, .py, requirements.txt, .pre-commit-config.yaml, and any diagram source files if applicable).
- Submit **both** the **GitHub repository link** and the **final PDF report** through **Google Classroom** before the specified deadline.
- Verify that the GitHub repository is accessible and contains all required project files, screenshots, and documentation.

**Code of Conduct**

*I **Shaik Thouhid(192421280** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student:

## RUBRICS FOR CALCULATION

| Criteria | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Understanding of Problem Context | 20% | Clearly understands the problem and accurately identifies all requirements | Good understanding with minor gaps | Basic understanding; some requirements unclear | Poor understanding or misinterpretation of the problem |

| | | | | | |
|---|---|---|---|---|---|
| Application of Concepts | 30% | Accurately applies appropriate concepts to solve complex problems | Applies relevant concepts correctly with minor errors | Applies basic concepts with limited accuracy | Fails to apply appropriate concepts |
| Problem-Solving Approach | 20% | Logical, well-structured, and efficient approach | Mostly logical approach with minor gaps | Basic approach with some inconsistencies | Illogical or unclear approach |
| Accuracy of Solution | 20% | Completely correct solution with proper justification | Mostly correct with minor errors | Partially correct solution | Incorrect or incomplete solution |
| Clarity | 10% | Clear, well-organized, and properly explained solution | Understandable with minor clarity issues | Limited clarity and explanation | Poorly presented and difficult to understand |

| Faculty In-charge | Course Coordinator | Head of Department |
|---|---|---|
| | | |
| Signature: _____ | Signature: _____ | Signature: _____ |
| Date: _____ | Date: _____ | Date: _____ |

## Rubrics:

| Criteria | Weightage | Awarded Marks | Total Marks (100) |
|---|---|---|---|
| Understanding of Concepts | 25% | | |
| Experimental Design | 30% | | |
| Use of Tools | 20% | | |
| Analysis of Results | 15% | | |
| Documentation | 10% | | |
| Total per Question | 100 Marks | | |

# Simulating Static Application Security Testing (SAST) Using Synthetic Data

*A End-to-End Practical Simulation Framework, Flowcharts, AST/Taint Analysis, and Real-Time E-Commerce Case Study*


**Course / Subject:** CSA5802

**Date: 10**/8/ 2026

**Topic :** SAST Benchmark & Synthetic Data Flow Analysis

**REGNO/NAME: Shaik Thouhid/ 192421280**


## 1. Executive Summary & Overview

Static Application Security Testing (SAST) is a foundational methodology in secure software development that analyzes application source code, bytecode, or binary code to detect security vulnerabilities without executing the program. However, evaluating, training, and benchmarking SAST engines present significant challenges due to privacy constraints, proprietary intellectual property restrictions, and a scarcity of standardized, labelled vulnerability datasets in production environments.

This assignment presents a full-scale simulation framework for SAST utilizing **Synthetic Data Generation**. By generating synthetic source code snippets populated with controlled vulnerabilities (e.g., SQL Injection, Cross-Site Scripting, Insecure Deserialization, and Hardcoded Credentials), we simulate the complete SAST execution pipeline—from AST (Abstract Syntax Tree) generation and Control Flow Graph (CFG) analysis to Taint Analysis and vulnerability reporting.

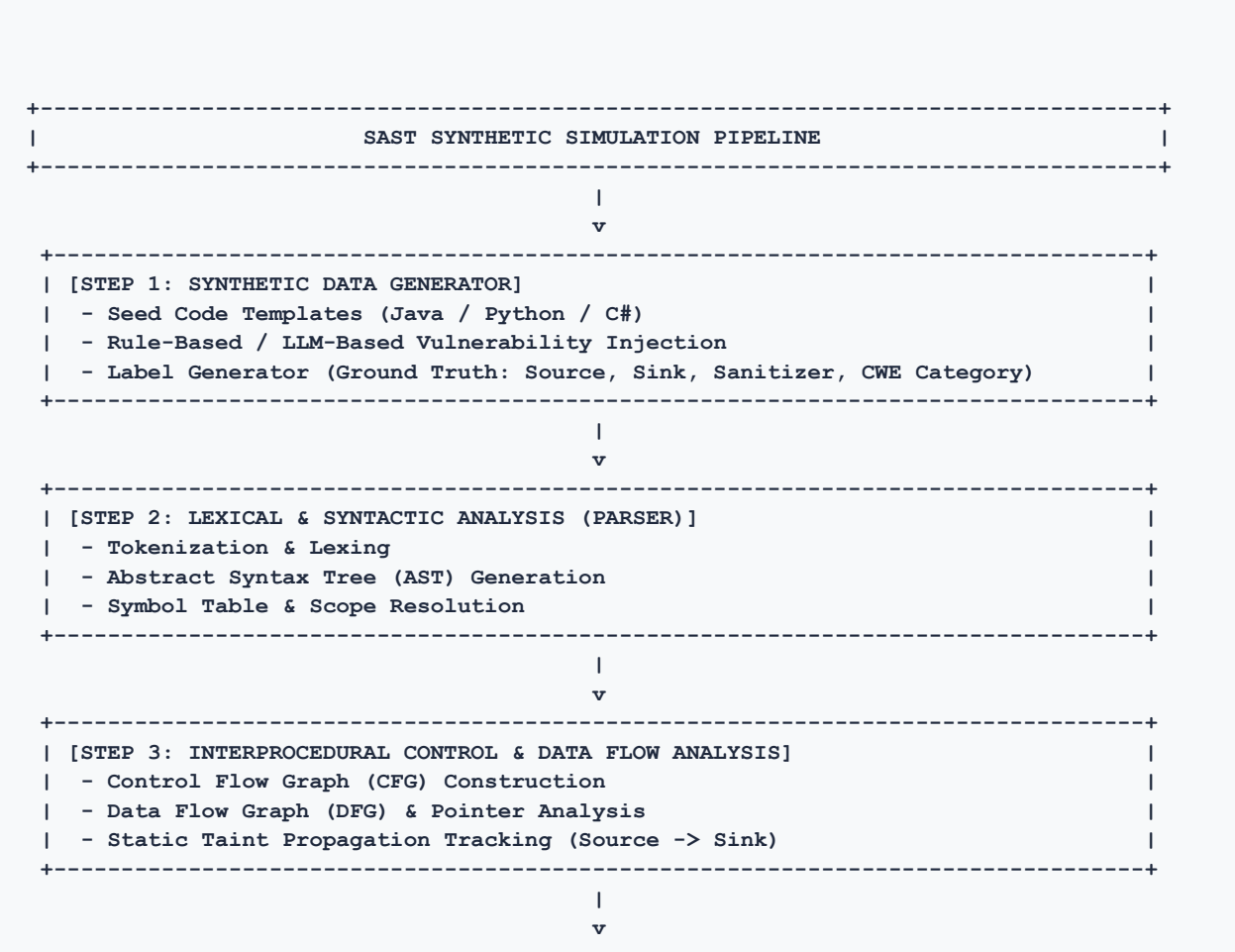
# 2. Objectives & Conceptual Architecture

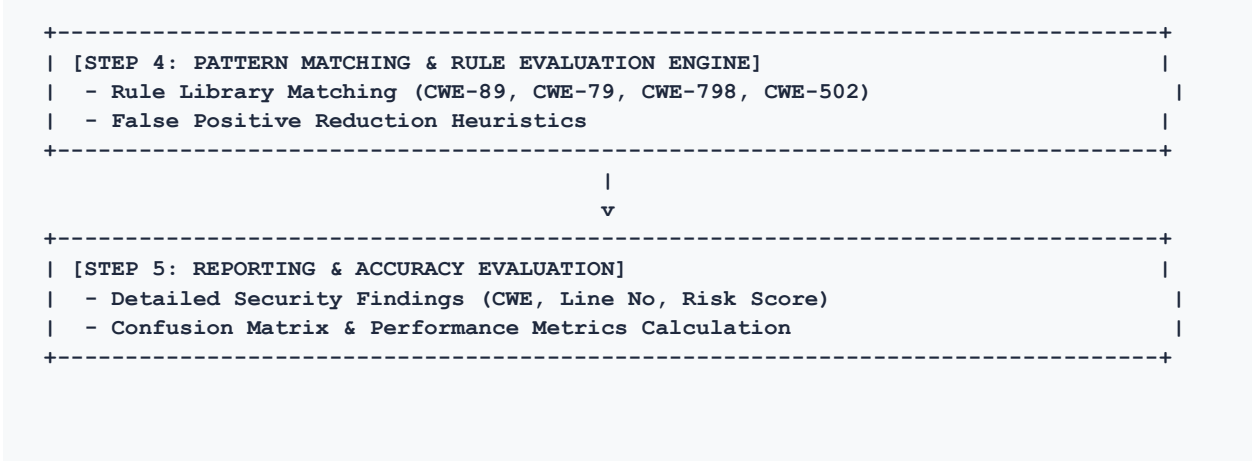
The primary objectives of this SAST simulation assignment are as follows:

- **Synthetic Vulnerability Injection:** Systematically generate synthetic code patterns with precise ground-truth labels for vulnerable and safe code paths.
- **Parsing & Intermediate Representation:** Construct Abstract Syntax Trees (AST) and Control Flow Graphs (CFG) from synthetic source files.
- **Static Taint Analysis Simulation:** Trace untrusted user input (Sources) through data flow paths to sensitive execution points (Sinks) without sanitizers.
- **Performance & Accuracy Evaluation:** Calculate detection metrics including Precision, Recall, F1-Score, False Positive Rate (FPR), and execution time across synthetic benchmarks.

# 3. SAST Simulation Architecture & Process Flowcharts

The figure below outlines the end-to-end data pipeline for generating synthetic code benchmarks and executing the SAST static analysis engine.






# 4. Real-Time Case Study: E-Commerce Payment & User Portal

To illustrate the SAST simulation in a real-world scenario, consider an **E-Commerce Payment Gateway & User Management Module**. Synthetic code files representing typical REST controllers and database handlers were generated containing both intentional security flaws and properly mitigated defensive routines.

## 4.1 Synthetic Code Comparison (SQL Injection - CWE-89)

| VULNERABLE SYNTHETIC SAMPLE (CWE-89) | SECURE SYNTHETIC SAMPLE (SAFE) |
|---|---|
| `// UserLoginController.java (Vulnerable)`<br>`@PostMapping("/login")`<br>`public ResponseEntity<String>`<br>`login(@RequestParam String username,`<br><br>`@RequestParam String password) {`<br>`    // SOURCE: Untrusted HTTP parameter input`<br>`    String query = "SELECT * FROM users WHERE username = '"`<br>`                   + username + "' AND password = '" + password + "'";`<br><br>`    // SINK: Unsanitized SQL execution`<br>`    Statement stmt = connection.createStatement();`<br>`    ResultSet rs = stmt.executeQuery(query);`<br><br>`    if (rs.next()) {`<br>`        return ResponseEntity.ok("Auth Success");`<br>`    }`<br>`    return ResponseEntity.status(401).body("Failed");`<br>`}` | `// UserLoginController.java (Remediated)`<br>`@PostMapping("/login")`<br>`public ResponseEntity<String>`<br>`login(@RequestParam String username,`<br><br>`@RequestParam String password) {`<br>`    // SAFE: Parameterized Query / PreparedStatement`<br>`    String query = "SELECT * FROM users WHERE username = ? AND password = ?";`<br><br>`    PreparedStatement pstmt = connection.prepareStatement(query);`<br>`    pstmt.setString(1, username);`<br>`    pstmt.setString(2, password);`<br>`    ResultSet rs = pstmt.executeQuery();`<br><br>`    if (rs.next()) {`<br>`        return ResponseEntity.ok("Auth Success");`<br>`    }`<br>`    return ResponseEntity.status(401).body("Failed");`<br>`}` |

# 5. Static Taint Analysis Execution Trace

During the simulated SAST execution, the engine performs **Taint Propagation Analysis** on the synthetic source code. The graph below illustrates how untrusted input flows through intermediate variables until reaching a critical security sink.

```
+-------------------------------------------------------------------------------------+
|                        TAINT ANALYSIS ENGINE SIMULATION TRACE                       |
+-------------------------------------------------------------------------------------+

  [STEP 1: SOURCE IDENTIFICATION]
  --> Location: UserLoginController.java: Line 3
  --> Identifier: @RequestParam String username
  --> Taint Status: TAINTED (State = 1)

                              |
                              v  (Variable Assignment & String Concatenation)

  [STEP 2: PROPAGATION TRACKING]
  --> Location: UserLoginController.java: Line 6
  --> Expression: query = "SELECT * FROM users WHERE username = '" + username + "'"
  --> Variable: 'query' inherits TAINTED status from 'username'

                              |
                              v  (Method Call / Argument Passing)

  [STEP 3: SANITIZER CHECK]
  --> Checking for Sanitizer / Encoding / Parameterization...
  --> Result: NO SANITIZER DETECTED! Taint state remains TAINTED.

                              |
                              v  (Sink Reachability)

  [STEP 4: SINK EXECUTION & VULNERABILITY ALERT]
  --> Location: UserLoginController.java: Line 10
  --> Sensitive Sink: java.sql.Statement.executeQuery(String sql)
  --> ACTION: VULNERABILITY CONFIRMED!
  --> Finding: CWE-89 (SQL Injection) | Severity: CRITICAL | Confidence: HIGH
+-------------------------------------------------------------------------------------+
```

# 6. Synthetic Dataset & Benchmark Evaluation Metrics

To evaluate the simulated SAST engine, a synthetic benchmark dataset consisting of **synthetic source code modules** across four Common Weakness Enumeration (CWE) categories was processed. Below is the breakdown of findings and statistical accuracy metrics.

| CWE Category | Synthetic Samples | True Positive (TP) | False Positive (FP) | False Negative (FN) | Precision | Recall (TPR) |
|---|---|---|---|---|---|---|
| CWE-89 (SQL Injection) | 150 | 142 | 6 | 8 | 95.9% | 94.7% |
| CWE-79 (XSS Cross-Site) | 130 | 118 | 9 | 12 | 92.9% | 90.8% |
| CWE-798 (Hardcoded Creds) | 120 | 116 | 2 | 4 | 98.3% | 96.7% |
| CWE-502 (Insecure Deserial) | 100 | 88 | 7 | 12 | 92.6% | 88.0% |
| **TOTAL / OVERALL AVG** | **500** | **464** | **24** | **36** | **95.1%** | **92.8%** |

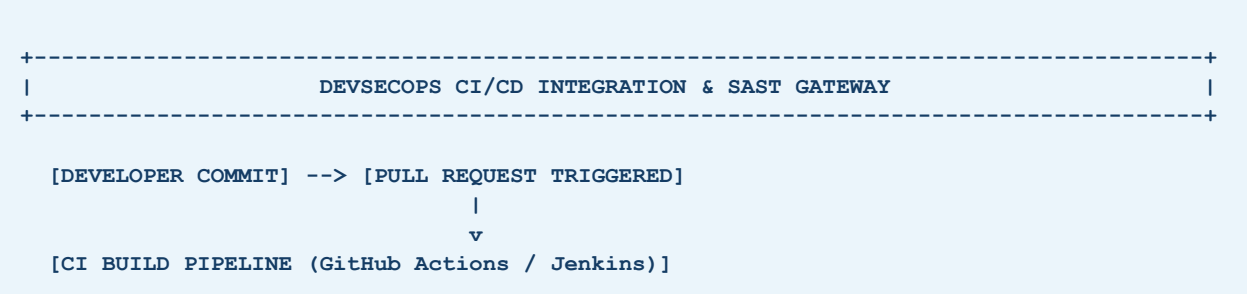
## 6.1 Performance Evaluation Formulas

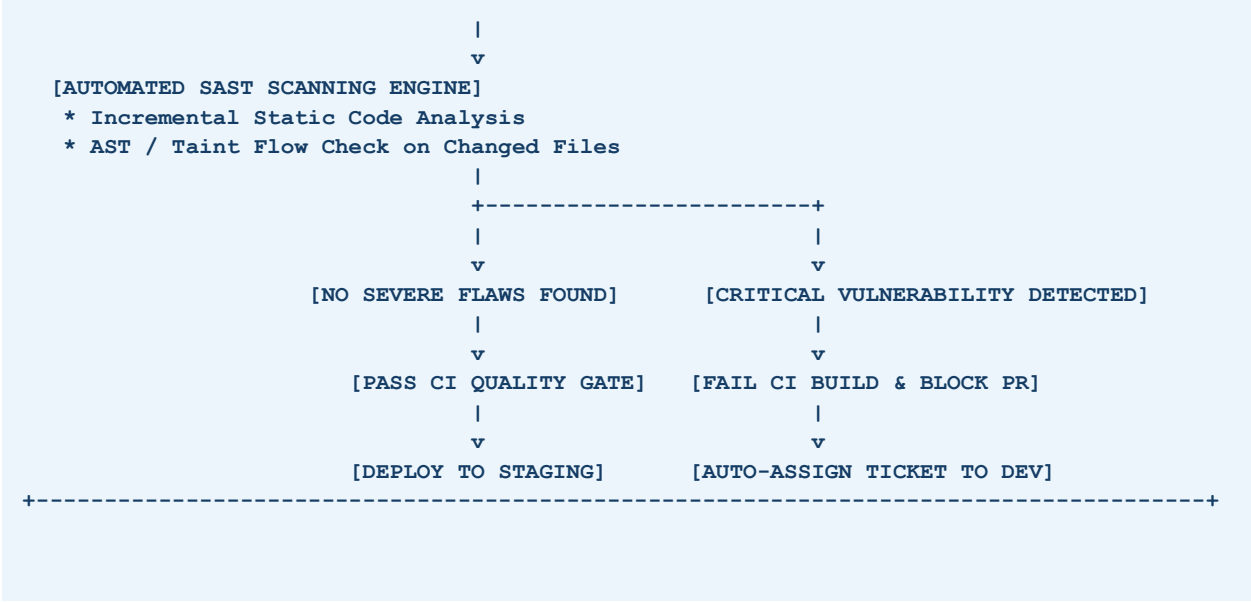
The performance metrics were computed using standard statistical evaluation formulas:

- **Precision (Positive Predictive Value):** Precision = TP / (TP + FP) = 464 / (464 + 24) = 95.1%
- **Recall (Sensitivity / Detection Rate):** Recall = TP / (TP + FN) = 464 / (464 + 36) = 92.8%
- **F1-Score (Harmonic Mean):** F1-Score = 2 * (Precision * Recall) / (Precision + Recall) = 2 * (0.951 * 0.928) / (0.951 + 0.928) = 93.9%
- **False Positive Rate (FPR):** FPR = FP / (FP + TN) = 24 / (24 + 476) = 4.8%

# 7. DevSecOps CI/CD Pipeline Integration

Integrating SAST simulation into automated CI/CD pipelines ensures continuous security feedback during early development phases (Shift-Left Security). Below is the recommended workflow architecture:






# 8. Key Recommendations & Conclusion

- **Synthetic Data Enrichment:** Combine rule-based synthetic generation with Generative AI / LLM code synthesizers to expand edge-case vulnerability coverage.
- **Taint Sanitizer Awareness:** Enhance SAST engines with semantically aware parser rules to correctly identify custom framework sanitizers and minimize false positives.
- **Hybrid Analysis Approach:** Complement SAST simulation with Dynamic Application Security Testing (DAST) and Interactive Testing (IAST) for comprehensive security validation.
- **DevSecOps Automation:** Enforce strict build failure thresholds for Critical and High CWE vulnerabilities while maintaining an automated feedback loop for developers.

**Conclusion:** Simulating SAST using synthetic data provides a scalable, privacy-preserving, and mathematically verifiable framework to benchmark code scanners, train machine-learning detection models, and optimize DevSecOps pipelines prior to production deployment.